Retrieval Mechanisms for RAG Verilog Generation

Miguel Carrillo
March 9, 2026

1 Introduction

This project focuses on hardware/Verilog generation using LLMs, since they could enable fast iteration from natural language specifications, reduced manual coding effort, and faster hardware design cycles.

The main problem is that vanilla LLMs perform poorly when generating Verilog hardware. Functional correctness measured with pass@1 remains below 60% even for the best models (mid-2024), as will be discussed further in the Relevant Prior Work section. In practice, vanilla LLMs exhibit:

- Low functional correctness for Verilog generation
- Weak structural reasoning for hardware tasks
- Poor pass@1 performance on Verilog benchmarks

Two main approaches have been proposed to improve performance.

Fine-tuning: Fine-tuning improves results but is costly, time-consuming, and often limited to open models.

Retrieval-Augmented Generation (RAG): RAG avoids fine-tuning by dynamically retrieving relevant examples (documentation or Verilog code) and appending them to the prompt. In RAG systems, two main retrieval strategies are used:

Vector Retrieval: Question → embedding → similarity search → top-k semantic examples.

Knowledge Graph Retrieval: Question → parameter extraction → KG filtering → top-k structural examples.

Prior work studies vector retrieval and hybrid approaches (vector + knowledge graph). However, knowledge graph retrieval has not been directly evaluated in isolation for Verilog generation. My key contribution will be to implement and evaluate knowledge graph retrieval for Verilog RAG. My research question is:

How does isolated knowledge graph retrieval compare to vector retrieval and hybrid approaches for Verilog generation?

This question is relevant because prior work shows that knowledge graph retrieval improves LLM performance in structured domains, suggesting potential benefits for hardware design. The main objectives of this project are:

- Replicate RAG with vector-based retrieval
- Replicate the hybrid RAG approach
- Implement a knowledge-graph-only retrieval method **(novel)**
- Compare the three approaches against a vanilla LLM baseline on the VerilogEval benchmark **(novel)**

2 Relevant Prior Work

2.1 LLMs for Verilog Generation

Recent work has explored using large language models (LLMs) to generate Verilog from natural language specifications. VerilogEval [5] introduced a benchmark with simulation-based testing, showing that vanilla LLMs achieve less than 60% pass@1 accuracy. A later study [6] reports improvements with newer models but confirms that Verilog generation remains challenging due to structural reasoning requirements.

Several works improve Verilog generation through domain-specific datasets and fine-tuned models. RTLCoder [9] and [7] introduce fine-tuned LLMs for RTL generation together with an open-source datasets.

2.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) improves LLM outputs by retrieving relevant external knowledge at inference time. DeepV [2] retrieves semantically similar Verilog examples using vector similarity search over a curated knowledge base. VeriRAG [8] extends this approach by combining it with knowledge graphs that filter structurally matching modules using extracted design parameters.

2.3 Knowledge Graph Retrieval

Knowledge graphs (KGs) represent structured relationships between entities and are widely used in machine learning systems as external symbolic memory [3]. StructuGraphRAG [10] demonstrates that graph-based representations improve retrieval and reasoning in structured technical documents. Recent work further shows that integrating knowledge graphs with retrieval-augmented generation improves reasoning accuracy in LLM systems [1, 4].

2.4 Gap in the Literature

Existing RAG approaches for Verilog generation evaluate vector-based retrieval or hybrid methods combining vector and graph retrieval. However, knowledge-graph-only

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

retrieval has not been evaluated in isolation, since systems such as VeriRAG[8] combine both mechanisms.

2.5 How This Project Builds on Prior Work

This project uses VerilogEval [5] for evaluation and the RTL-Coder dataset [9] for retrieval examples. Inspired by VeriRAG [8], this work implements knowledge graph retrieval and evaluates it independently against vector-based and hybrid RAG approaches.

3 Methodology

3.1 Approach Overview

This project evaluates three retrieval strategies for Retrieval-Augmented Generation (RAG) in Verilog code generation: vector-based retrieval, knowledge-graph (KG) retrieval, and a hybrid combination of both. These methods will be compared against a vanilla LLM baseline without retrieval.

For each method, relevant Verilog examples are retrieved from an external corpus and appended to the prompt before generation. The goal is to determine whether structurally informed retrieval via knowledge graphs improves generation quality compared to semantic vector retrieval.

3.2 Datasets and Benchmarks

Evaluation will be conducted using the VerilogEval benchmark [5], which contains natural language hardware specifications paired with simulation-based testbenches. VerilogEval automatically compiles and simulates generated Verilog code to determine functional correctness.

For retrieval, the RTL-Coder corpus[9] will be used. It contains diverse hardware modules and will serve as the knowledge base for the RAG system.

3.3 Simulation Environment

Functional correctness will be evaluated using the Icarus Verilog simulator (iverilog). Generated modules will be compiled and executed against the VerilogEval testbenches.

3.4 Retrieval System Implementation

The project will implement two retrieval mechanisms:

Vector Retrieval. A vector database will store embeddings of Verilog modules and their associated descriptions. Given a natural language specification, the system retrieves the top-k semantically similar examples using embedding similarity search. These examples are appended to the prompt provided to the language model.

Knowledge Graph Retrieval. A knowledge graph will represent structural attributes of hardware modules, such as module type, number of inputs and outputs, parameterization, and functional category. Query specifications will be parsed to extract relevant design parameters, which will be used to filter the graph and retrieve structurally similar modules.

Initially, the parameter schema proposed in VeriRAG [8] will be adopted, with potential extensions to improve robustness of parameter extraction.

3.5 RAG Pipeline Construction

The complete RAG pipelines will be implemented incrementally. First, a baseline system using a vanilla LLM without retrieval will be evaluated on the benchmark. Next, vector-based retrieval will be integrated, followed by the knowledge-graph retrieval system. Finally, a hybrid approach combining both retrieval mechanisms will be implemented.

Initial experiments will use open-source language models to verify correctness of the pipeline and reduce computational cost. Once the system is validated, the experiments will be repeated using GPT-3.5 to allow direct comparison with results reported in prior work.

3.6 Evaluation Protocol

All configurations will be evaluated using the same benchmark and generation settings. For each task in VerilogEval, the language model will generate a single Verilog implementation conditioned on the retrieved examples (if any). The generated code will then be compiled and simulated using the provided testbenches.

3.7 Success Metrics

Performance will be measured using **pass@1**. It measures the percentage of generated programs that pass all functional test cases on the first attempt. Comparing pass@1 across the four configurations will allow us to quantify the impact of different retrieval strategies on Verilog generation performance.

4 Timeline

The planned schedule for the project is shown in Table 1.

Table 1. Project timeline

Week	Planned Work
W1 (Feb 28–Mar 8)	Project Proposal
W2 (Mar 9–Mar 15)	Dataset Preparation
W3 (Mar 16–Mar 22)	Build Retrieval Systems
W4 (Mar 23–Mar 29)	Implement & Evaluate Vanilla & Vector
W5 (Mar 30–Apr 5)	Implement & Evaluate KG
W6 (Apr 6–Apr 12)	Implement & Evaluate Hybrid [Presentation Deadline]
W7 (Apr 13–Apr 19)	Write and send first report draft [First Report Draft Deadline]
W8 (Apr 20–Apr 26)	Finish experimentation and results analysis; write final report
W9 (Apr 27–Apr 30)	[Project Report Deadline]

References

[1] Y. Huang and G. Zeng. 2024. RD-P: A Trustworthy Retrieval-Augmented Prompter with Knowledge Graphs for Large Language Models. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. doi:10.1145/3627673.3679659

[2] Zarin Ibnat, Pedro E. Calzada, R. M. Ihtemam, S. K. Saha, J. Zhou, Farimah Farahmandi, and Mark Tehranipoor. 2025. DeepV: A Model-Agnostic Retrieval-Augmented Framework for Verilog Code Generation with a High-Quality Knowledge Base. *arXiv preprint arXiv:2510.05327* (2025). https://arxiv.org/abs/2510.05327

[3] Shaoxiong Ji, Shirui Pan, Erik Cambria, Pekka Marttinen, and Philip S. Yu. 2022. A Survey on Knowledge Graphs: Representation, Acquisition and Applications. *IEEE Transactions on Neural Networks and Learning Systems* (2022). doi:10.1109/TNNLS.2021.3070843

[4] J. Linders and J. M. Tomczak. 2025. Knowledge Graph-Extended Retrieval-Augmented Generation for Question Answering. *Applied Intelligence* (2025). doi:10.1007/s10489-025-06885-5

[5] Mingjie Liu, Nathan Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. https://arxiv.org/abs/2309.07544

[6] Mingjie Liu, Nathan Pinckney, Brucek Khailany, and Haoxing Ren. 2024. Revisiting VerilogEval: A Year of Improvements in Large Language Models for Hardware Code Generation. *arXiv preprint arXiv:2408.11053* (2024). https://arxiv.org/abs/2408.11053

[7] Shuo Liu, Yifan Lu, Wei Fang, Meng Li, and Zhiyao Xie. 2024. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation. In *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. doi:10.1145/3676536.3697118

[8] R. Saravanan et al. 2024. VeriRAG: Knowledge Graph-Augmented Retrieval-Augmented Generation for Verilog and Assertions. In *Proceedings of the Design Automation Conference (DAC)*. https://mason.gmu.edu/~rsaravan/papers/VeriRAG.pdf

[9] Zhiyao Xie, Shuo Liu, Wei Fang, Yifan Lu, and Qiang Zhang. 2024. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024). https://zhiyaoxie.com/files/TCAD25_RTLCoder.pdf

[10] Xiaohui Zhu, Xiaotong Guo, Shuaichen Cao, Shuo Li, and Jian Gong. 2024. StructuGraphRAG: Structured Document-Informed Knowledge Graphs for Retrieval-Augmented Generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. https://ojs.aaai.org/index.php/AAAI-SS/article/view/31798